

Lista de Exercícios para Primeira avaliação

Avalei e responda as questões:

Com base nos materiais fornecidos, preparei uma lista de 50 exercícios que abrangem desde os conceitos iniciais da linguagem C até as estruturas de dados dinâmicas (LSE e LDE), incluindo depuração e aplicações práticas.

Parte 1: Conceitos Iniciais e Repetição (Aulas 1 e 2)

1. Explique a diferença entre a **sintaxe** e a **semântica** de uma linguagem de programação como o C.
2. Descreva como funciona o **operador ternário** `?` `:` e dê um exemplo de uso para substituir uma instrução `if-else`.
3. O que caracteriza a **tipagem forte** na linguagem C durante a declaração de variáveis?
4. Diferencie o funcionamento das estruturas de repetição `while` e `do-while`, destacando qual delas garante ao menos uma execução.
5. Escreva o comando necessário para compilar um arquivo chamado `Teste.c` utilizando o compilador **GCC** no Linux.

Parte 2: Ponteiros e Vetores (Aula 2)

6. Defina o que é um **ponteiro** e qual a função dos operadores `&` (endereço) e `*` (indireção/valor).
7. O que significa atribuir o valor **NULL** a um ponteiro?
8. Explique a diferença entre **passagem por valor** e **passagem por referência** em funções, utilizando o exemplo da troca de valores entre duas variáveis.
9. Como a memória RAM é organizada em relação a **endereços** e **dados**?
10. Por que em C dizemos que **vetores e strings são ponteiros**?
11. Implemente uma função `SomaAB` que receba dois inteiros e um ponteiro para armazenar o resultado da soma.
12. Crie uma função `menorVetor` que utilize ponteiros para encontrar o menor valor em um vetor de inteiros.
13. Como funciona a **aritmética de ponteiros** ao incrementar um ponteiro que aponta para um vetor?
14. Explique o conceito de **ponteiro de ponteiro** (`**ppx`) e como ele permite alterar o endereço para o qual um ponteiro aponta.
15. Escreva uma função que receba uma string e conte a quantidade de vogais utilizando apenas navegação por ponteiros.

Parte 3: Alocação Dinâmica e Structs (Aulas 2 e 3)

- 16.Qual a função do comando **malloc** e por que é necessário usar o operador **sizeof**?
- 17.Por que é fundamental utilizar a função **free** após a alocação dinâmica de memória?
- 18.Como o comando **typedef** melhora a legibilidade ao trabalhar com **structs**?
- 19.Explique o conceito de **structs aninhadas** (composição), exemplificando com as estruturas **Pessoa** e **Endereco**.
- 20.Qual a diferença de sintaxe para acessar um membro de uma **struct** via **variável** (ponto **.**) e via **ponteiro** (seta **->**)?
- 21.Implemente uma **struct Carro** que contenha uma **struct Motor** interna (potência, modelo, consumo).
- 22.Crie uma função que aloque dinamicamente uma estrutura **Apartamento** e retorne seu ponteiro.
- 23.Em um sistema de festa, como você associaria um **Convidado** a um **Convite** utilizando ponteiros?
- 24.Explique como criar um **vetor de ponteiros para structs**, onde cada elemento do vetor aponta para uma área de memória alocada individualmente.
- 25.Desenvolva uma função **venderApartamento** que receba o ponteiro de um apartamento e o ponteiro de um novo proprietário, realizando a troca de dono.

Parte 4: Listas Simplesmente Encadeadas - LSE (Aula 4 e Lista 1)

- 26.Defina o que é um **Nó (Elemento)** de uma lista e quais são seus dois componentes básicos.
- 27.Diferencie **alocação estática** (vetores) de **alocação dinâmica** (listas) em termos de tamanho e acesso.
- 28.Explique o papel do ponteiro ***primeiro** em uma estrutura de gerenciamento de lista LSE.
- 29.Descreva o algoritmo para **inserir um elemento no início** de uma LSE, considerando os casos de lista vazia e com elementos.
- 30.Por que para **inserir no fim** de uma LSE sem ponteiro para o último elemento é necessário usar um laço **while**?
- 31.Como é identificado o **fim de uma LSE**?
- 32.**Depuração:** O que acontece se você tentar inserir um novo nó no início, mas esquecer de atualizar o ponteiro **proximo** do novo nó para o antigo **primeiro**?
- 33.**Depuração:** Por que a função **free(aux)** é necessária em uma remoção, e o que ocorre se ela for omitida?
- 34.Implemente uma função para **contar a quantidade de elementos** em uma LSE percorrendo-a até o fim.
- 35.Como funciona a **remoção no início** de uma LSE? Descreva a movimentação do ponteiro **primeiro**.
- 36.Qual o risco de tentar acessar **atual->proximo** em um laço de busca se o ponteiro **atual** for **NULL**?
- 37.Escreva uma função que mostre todos os alunos de uma lista navegando pelo ponteiro **proximo**.
- 38.Explique a lógica necessária para **inserir um elemento em uma posição específica (N)** de uma LSE.

Parte 5: Listas Duplamente Encadeadas - LDE (Aula 5 e Lista 2)

39. Qual a principal diferença estrutural entre uma LSE e uma **LDE**?
40. Explique a função dos ponteiros **anterior** e **proximo** em um nó de LDE.
41. Qual a vantagem de manter um ponteiro para o **último** elemento em uma LDE?
42. Descreva como realizar a **mostra da lista da direita para a esquerda** (ordem inversa) em uma LDE.
43. Como deve ser feita a **inserção no início** de uma LDE quando a lista já possui outros elementos? (Dica: atualização do ponteiro **anterior**).
44. Explique a **inserção otimizada** no fim de uma LDE utilizando o ponteiro **ultimo** (Complexidade $\theta(1)$).
45. Implemente a lógica para **remover o último elemento** de uma LDE, garantindo que o ponteiro **ultimo** do gerenciador seja atualizado.
46. Em um sistema de **atendimento bancário**, como você usaria uma LSE para registrar o histórico (log) após remover um cliente de uma fila?
47. Como identificar se uma **LDE está vazia** verificando seus ponteiros de controle?
48. **Aplicação:** No sistema de vacinação de cães, como uma LSE poderia armazenar as múltiplas vacinas aplicadas em um único animal?
49. Qual o perigo de liberar a memória de uma lista (**apagaLista**) sem percorrer nó por nó para dar **free**?
50. Explique o conceito de **lista sem cabeça** (sem nó sentinela) e quais cuidados devem ser tomados na inserção no início.

